TinySQL: A 125M-Parameter Text-to-SQL Model Built and Pretrained From Scratch on 4.5B Tokens

Aditya Jha
Independent Researcher
https://github.com/aditya0929/tinysql

October 2026

Abstract

We describe TinySQL-125M, a decoder-only language model for text-to-SQL in which every component of the model path is written from scratch: a byte-level BPE tokenizer, the transformer itself (grouped-query attention, rotary embeddings, SwiGLU, RMSNorm), the optimizer, the training loop, the data pipeline, a grammar-constrained decoder and the evaluation harness. The model is pretrained for about 46 hours on a single NVIDIA L4 GPU on 4.5B tokens of educational web text and SQL code, then fine-tuned on WikiSQL. To make the result interpretable, we fine-tune four public models of similar size (GPT-2 124M, Pythia-160M, SmolLM2-135M and SmolLM2-360M) with an identical recipe, sweep the learning rate for every model, average over three seeds, train our architecture without pretraining as a control, and report confidence intervals and paired significance tests throughout. On the WikiSQL test set, TinySQL reaches $77.95 \pm 0.18\%$ execution accuracy with greedy decoding and $80.39 \pm 0.20\%$ with schema- and value-constrained decoding (official evaluator: 78.03% and 80.50%). This is about one point below GPT-2 and 17 points above Pythia-160M, which was pretrained on 67× more tokens; SmolLM2-135M, pretrained on 2T tokens (about 440× more), is 4.6 points ahead. Pretraining accounts for 41 points of accuracy. Constrained decoding never turned one of our model's correct answers into a wrong one in any paired comparison we ran. Small-scale ablations show that rotary embeddings and tied embeddings matter far more than grouped-query attention or an SQL-heavy pretraining mix, and a hand audit suggests that most remaining aggregation errors are label errors in WikiSQL. All code, results and the 166 tests that verify the components are public.

1 Introduction

Small language models are attractive for text-to-SQL: they run on a CPU, can be deployed next to a database, and are cheap to fine-tune. Most small models available today are pretrained by well-resourced teams on trillions of tokens. This report asks a narrower question: *how far can a single person get on a standard text-to-SQL benchmark with a model whose every part they built themselves, on a pretraining budget of a few billion tokens and one rented GPU?*

To answer it credibly, the system is built without any pretrained weights, pretrained tokenizers or high-level modeling libraries (Section 3), and the evaluation is designed so that differences can be attributed (Section 4): every baseline gets the same fine-tuning recipe and its own learning-rate sweep, main results are averaged over seeds, the test set is used once, and comparisons are paired and tested for significance.

Contributions.

- A complete, tested, from-scratch text-to-SQL stack: tokenizer, Llama-style model [22], optimizer, a data pipeline with deduplication and benchmark decontamination, a grammar-constrained decoder, and evaluation code (Section 3).
- A controlled comparison on WikiSQL [27] against four public models of similar size, with learning-rate sweeps, three seeds, a no-pretraining control, bootstrap confidence intervals and McNemar tests (Section 5).
- Evidence that a 4.5B-token model trained on a deliberate web+SQL mix comes within about one point of GPT-2 and well above Pythia-160M at this task, and that constrained decoding helps weaker models more without ever hurting ours.
- Ablations of four design choices (Section 5.3), a failure analysis, and a hand audit of aggregation labels (Section 5.4).

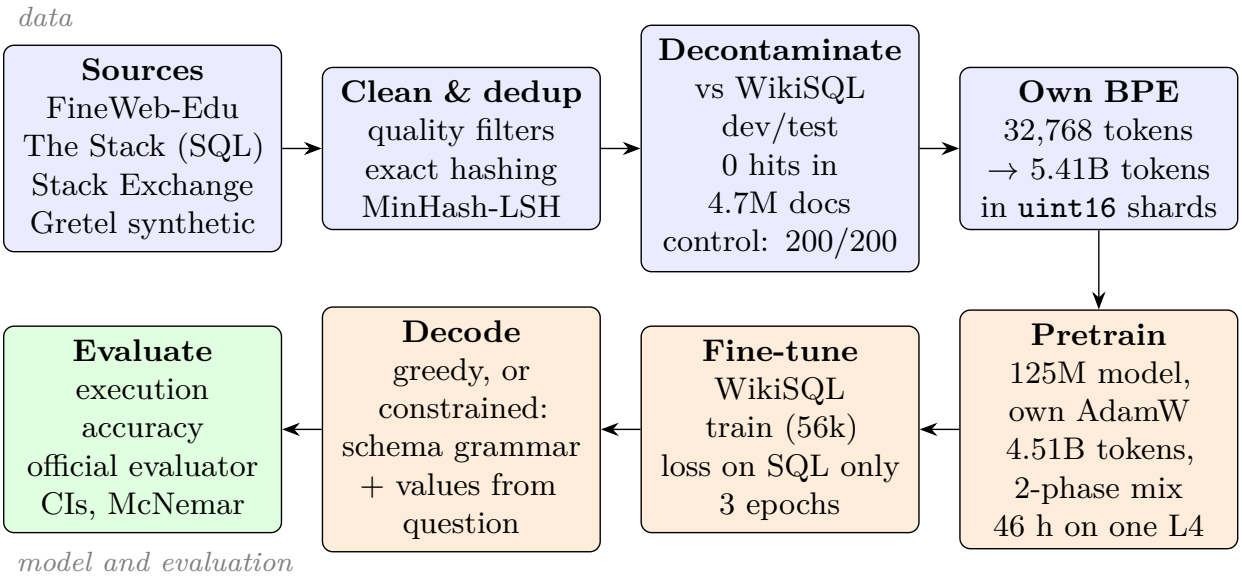


Figure 1: **End-to-end pipeline.** Every stage is implemented in this project; pretrained weights and tokenizers are used only for the baselines.

2 Related work

WikiSQL and text-to-SQL. WikiSQL [27] pairs 80,654 crowd-written questions with SQL queries over single Wikipedia tables. Its queries select one column, optionally aggregate it, and filter with up to four conjunctive conditions. Strong specialised systems built on large pretrained encoders report test execution accuracy in the high 80s, and their authors attribute a large share of the remaining errors to annotation errors [9]; recent work cleans the dataset for the LLM era [15]. Multi-table benchmarks such as Spider [25] are harder and are not addressed here.

Layers / hidden size	30 / 576
Query heads / KV heads	9 / 3 (head dim 64)
MLP	SwiGLU, width 1536
Positions	RoPE, $\theta = 10{,}000$, context 2048
Normalisation	RMSNorm, pre-norm
Embeddings	tied input/output, vocabulary 32,768
Parameters	125,077,824

Table 1: **Model hyperparameters** of TinySQL-125M. The parameter count is derived by hand and asserted in a test.

Constrained decoding. Restricting generation to well-formed SQL is an established technique, either by checking partial queries against a parser (PICARD [18]) or by executing candidate queries (execution-guided decoding [23]). We use a simple byte-level grammar specialised to the WikiSQL query shape, with a stricter level that only admits values copied from the question.

Small open models. GPT-2 [17], Pythia [3] and SmolLM2 [2] are open models in the 100–400M range with very different pretraining budgets (Pythia: 300B tokens; SmolLM2-135M/360M: 2T/4T tokens; GPT-2: not published). Our architecture follows Llama [22]: RoPE [21], RMSNorm [26], SwiGLU [20], grouped-query attention [1] and tied embeddings [16].

3 System

3.1 What "from scratch" means here

All model code is written by hand on top of PyTorch tensors and autograd: linear and embedding layers, RMSNorm, rotary embeddings, grouped-query attention with a KV cache, SwiGLU, a memory-efficient cross-entropy, sampling, AdamW and the training loop. No pretrained weights, pretrained tokenizers or modeling classes from `transformers` are used anywhere in our model's path. Hugging Face code appears in three roles only: as a test oracle (our weights loaded into a reference Llama implementation produce the same logits to within 10^{-4}, for the full 125M model), to download datasets, and to run the baselines. PyTorch's fused attention kernel is an optional backend that is tested against our own attention implementation. The repository contains 166 tests, including exact-resume of training, optimizer agreement with `torch.optim.AdamW` to 10^{-6}, and a positive control for decontamination.

3.2 Tokenizer

A byte-level BPE [19] tokenizer with 32,768 tokens (256 bytes, 32,500 merges, 12 special tokens such as `<schema>`, `<question>` and `<sql>`). It was trained on 955 MB of text deliberately weighted towards SQL, plus WikiSQL *training* questions and column names. Common SQL keywords become single tokens (␣`SELECT`, ␣`FROM`, ␣`WHERE`). Held-out compression is 4.49 bytes/token on FineWeb-Edu and 3.60 bytes/token on the median SQL file. Web, code and forum text is encoded with special tokens disabled, so crawled text containing the literal string `<sql>` cannot inject a control token.

3.3 Model

Table 1 and Figure 2 give the architecture. Weights are initialised from $\mathcal{N}(0, 0.02)$ with residual output projections scaled by $1/\sqrt{2L}$. The loss is a custom autograd function that stores only the

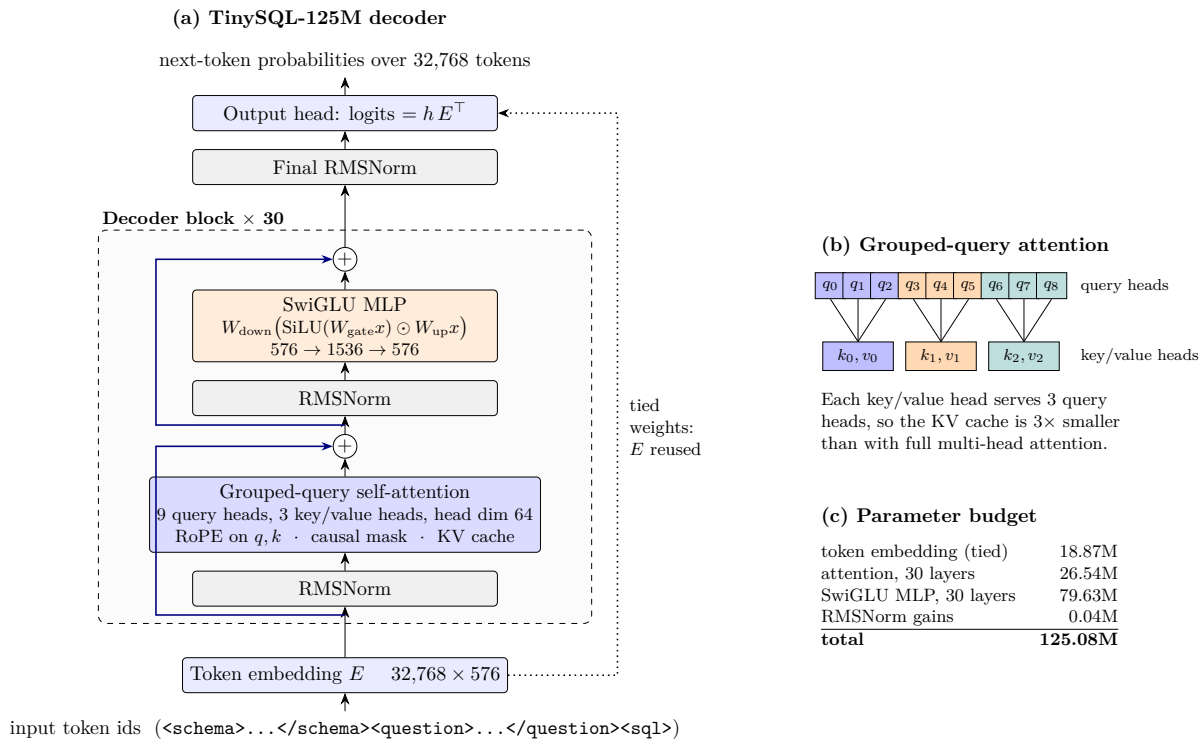


Figure 2: **Model architecture.** (a) The Llama-style pre-norm decoder: 30 identical blocks between a token embedding and an output head that reuses the embedding matrix (tied weights); blue lines are residual connections. (b) Grouped-query attention: each of the 3 key/value heads serves 3 of the 9 query heads. (c) Parameter budget by component.

Source	Documents	Tokens	Share of shards
FineWeb-Edu [14]	3,589,789	3.84B	71.0%
The Stack, SQL subset [10]	612,761	1.42B	26.3%
Stack Exchange (dba, datascience)	122,659	0.12B	2.2%
Gretel synthetic text-to-SQL [7] (×3)	99,164	0.025B	0.5%
Total		5.41B	

Table 2: **Pretraining corpus:** shards after cleaning, deduplication and decontamination. The mix actually trained on is set by sampling weights (Section 3.5).

logits and one log-sum-exp per token, which was needed to fit micro-batches of 8×2048 tokens with a 32,768-way output on a 24 GB GPU.

3.4 Pretraining data

Table 2 lists the sources. The pipeline applies per-source quality filters (for example, `INSERT`-heavy SQL dumps are truncated to keep the schema and drop bulk data rows), exact deduplication across all sources, and MinHash-LSH near-duplicate removal [4] for SQL files. From the SQL files, the 70% with the most query features (`SELECT`, `JOIN`, `GROUP BY`, aggregates, and so on) are kept.

Decontamination. Every pretraining document was scanned for WikiSQL dev and test questions: any shared 13-gram for questions of 13 or more words, or the whole question for questions of 8 to 12 words. No document matched in 4.7M scanned. Because a scanner that finds nothing proves little by itself, we ran a positive control through the same code: 200 dev/test questions planted into 1,000 real documents were all flagged, with no false positives. Questions shorter than 8 words (14%) are too generic to scan and gold SQL strings were not scanned; both are limitations.

3.5 Pretraining

We train with our own AdamW [12] ($\beta = (0.9, 0.95)$, weight decay 0.1 on weight matrices only), gradient clipping at 1.0, bf16 autocast and `torch.compile`. The peak learning rate of 1.2×10^{-3} comes from a sweep on a 25M-parameter proxy (optimum near 1.4–1.5×10^{-3}, lowered for the larger model and longer run), with 200 warmup steps and cosine decay to 10%. Each step is 524,288 tokens; the run is 8,600 steps, i.e. 4.51B tokens or about 3.4×10^{18} FLOPs, roughly $1.8\times$ the compute-optimal token count for this size [8]. Data is mixed in two phases (Figure 3): 82/15/2.5/0.5% (web/SQL/Stack Exchange/Gretel) for the first 85% of steps, then 54/36/8/2% while the learning rate decays, so the specialised data is concentrated where the learning rate is low.

The run took about 46 hours on one NVIDIA L4 at 27k tokens/s (about 17% model FLOPs utilisation), with zero skipped steps. Final validation loss is 2.94 nats/token on FineWeb-Edu and 0.96 on SQL code; every source improved at every checkpoint (Figure 4).

3.6 Fine-tuning and decoding

Each WikiSQL example becomes a prompt with the table schema as a `CREATE TABLE` statement and the question, and a completion with the SQL; the loss is computed on the completion only:

```
<schema>CREATE TABLE t ("Player" TEXT, "No." TEXT, "Nationality" TEXT, "Position" TEXT, "
    Years in Toronto" TEXT, "School/Club Team" TEXT);</schema>
<question>What position does the player who played for butler cc (ks) play?</question>
```

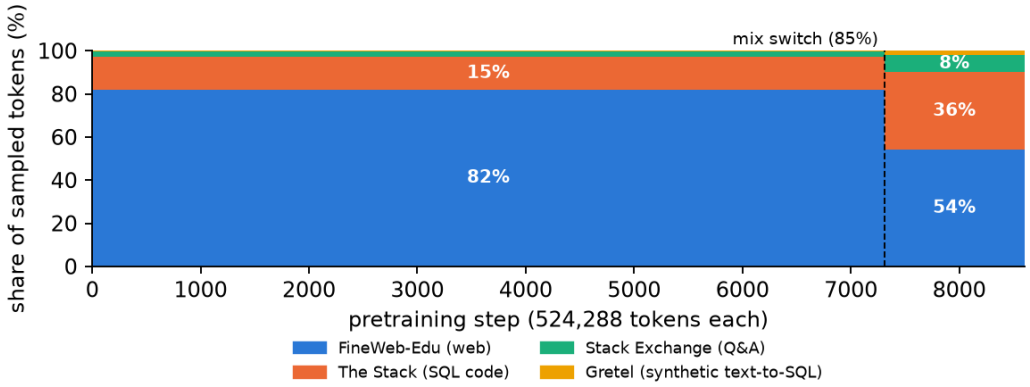


Figure 3: **Pretraining data mix:** share of sampled tokens per source over the run. Sources are sampled at training time from fixed shards, so the mix can change without re-tokenising.

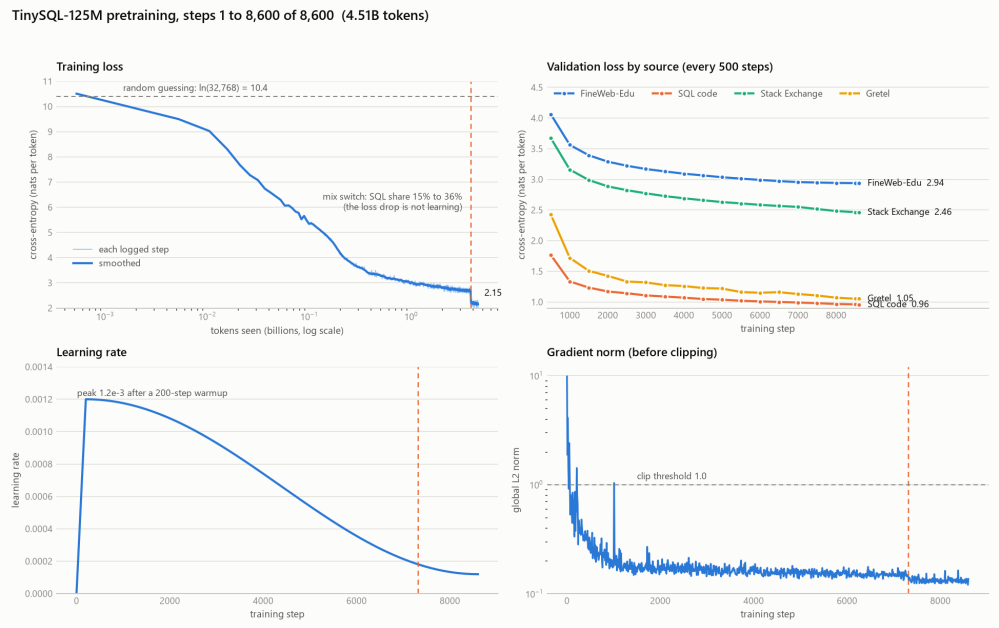


Figure 4: **Pretraining curves:** training loss, validation loss per source, learning rate and gradient norm over the 8,600 steps. The drop in training loss at the mix switch (step 7,310) reflects the easier SQL-heavy mix and was predicted from the per-source losses beforehand.

```
<sql>SELECT "Position" FROM t WHERE "School/Club Team" = 'Butler CC (KS)';</sql>
```

Fine-tuning uses batch 32, 100 warmup steps, cosine decay, no weight decay and 3 epochs (5,283 steps, about 30 minutes on the L4). After each epoch, execution accuracy is measured on the first 1,000 dev questions and the best epoch is kept.

Constrained decoding. Two facts about WikiSQL motivate it. 95% of our model's invalid greedy outputs reference a column that does not exist, and 100% of gold condition values (11,545 of 11,545 on dev) occur verbatim, ignoring case, in the question. We therefore define two levels. *Schema* is a byte-level grammar for the WikiSQL query shape that only admits the table's real column names. *Schema + values* additionally requires every condition value to be a substring of the question (Unicode-aware, valid UTF-8). At each step, candidate tokens are checked best-first and the first that keeps the output a valid prefix is taken, which is equivalent to masking every invalid token and taking the argmax (Figure 5). The same decoder runs on the baselines through a byte-level adapter for their tokenizers.

Columns: `"Player"`, `"No."`, `"Nationality"`, `"Position"`, `"Years in Toronto"`, `"School/Club Team"`

Question: What position does the player who played for butler cc (ks) play?

`SELECT "` | → `Role` `Pos.` `Position`

schema level: only prefixes of real column names pass

`..."School/Club Team" = '` | → `Butler College` `butler cc`

values level: a condition value must also be a substring of the question

Figure 5: **Constrained decoding** on a WikiSQL dev question (candidate tokens are illustrative). Candidates are tried left to right in model-score order: red ones would leave the grammar and are skipped, green is the first valid one. Checking is lazy, so usually only a few candidates are tested; the result equals masking every invalid token and taking the argmax.

4 Experimental setup

Baselines. GPT-2 124M [17], Pythia-160M [3], SmolLM2-135M and SmolLM2-360M [2], fine-tuned with our recipe: the same prompt text, loss masking, batch size, optimiser (our AdamW), schedule, epochs and epoch selection. For memory, the baselines accumulate gradients over token-weighted micro-batches; a test shows this gives the same gradient as the full batch to 10^{-6}. As a control, our architecture is fine-tuned from random weights with the same recipe.

Protocol.

1. **Learning rate**, per model, from $\{5 \times 10^{-5}, 10^{-4}, 2 \times 10^{-4}\}$ (control: $\{10^{-4}, 10^{-3}\}$), chosen on the first 1,000 dev questions (Figure 6; Table 7 in the appendix).
2. **Seeds:** three for TinySQL and both SmolLM2 models; one for GPT-2, Pythia and the control.

3. **Test set, once:** the dev-phase weights had been deleted after scoring, so every system was fine-tuned again with its dev-chosen configuration and scored on test exactly once. No decision was made after seeing a test number.

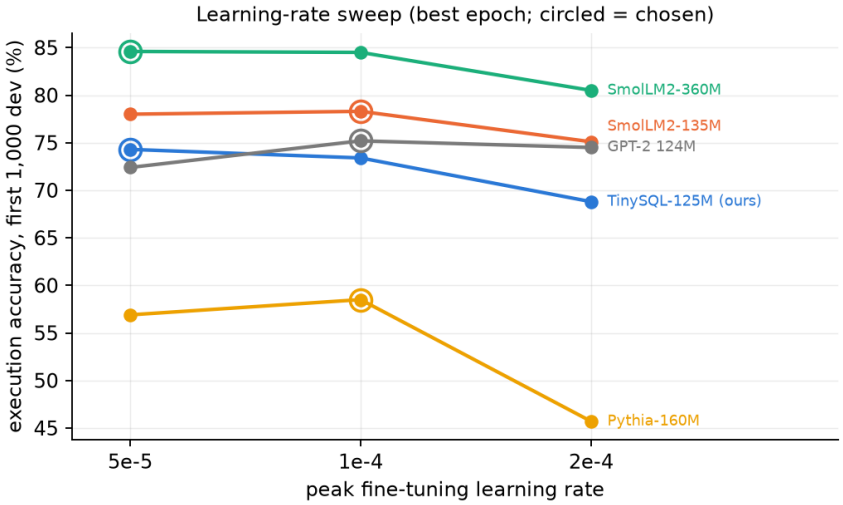


Figure 6: **Learning-rate sweep** for fine-tuning, seed 0. Every model gets its own best learning rate; 2×10^{-4} is too high for all of them.

Metrics. Our main metric is execution accuracy: the predicted and gold queries are executed on the table, in a sandboxed read-only SQLite connection, and their results compared. We also run Salesforce's official WikiSQL evaluator, unmodified, which reports execution and logical-form accuracy. Fed the gold queries as predictions, it scores exactly 100% on dev and test, and its execution accuracy agrees with ours within 0.11 points on every run of the four main systems.

Statistics. For each system we report the mean and standard deviation over seeds and a 95% percentile-bootstrap confidence interval [6] over questions, using 10,000 resamples of the seed-averaged per-question accuracy. Comparisons between systems pair their seed-0 predictions question by question, using a paired bootstrap [11] and the exact McNemar test [13] on the questions where exactly one system is correct, following [5].

5 Results

5.1 Main results

Table 3 gives test accuracy and Table 4 the paired comparisons. Figure 7 places each model by its pretraining tokens.

1. **Pretraining is most of the result.** The same network fine-tuned from random weights reaches 37.4%. Pretraining on 4.5B tokens adds 40.6 points, and constrained decoding cannot substitute for it (48.3% even with both constraints).

Model	Pretraining tokens	Seeds	Greedy	+ Schema	+ Schema + values
TinySQL-125M (ours)	**4.5B**	3	**77.95** ± **0.18**	79.06	**80.39** ± **0.20**
GPT-2 124M	unknown	1	78.93	80.07	82.36
Pythia-160M	300B	1	60.78	63.98	67.63
SmolLM2-135M	2T	3	82.57 ± 0.37	83.16	85.22 ± 0.21
SmolLM2-360M	4T	3	87.03 ± 0.19	87.42	88.01 ± 0.12
Ours, no pretraining	0	1	37.39	–	48.33

Table 3: **Main results:** WikiSQL **test** execution accuracy (%, 15,878 questions). Mean ± std over seeds; schema-only uses seed 0. 95% bootstrap CIs are ±0.5 to 0.8 points (Table 9 and the repository). Dev results (Table 8) agree closely.

Comparison (test, greedy)	Difference [95% CI]	Discordant	p (McNemar)
SmolLM2-360M − ours	+9.20 [+8.6, +9.8]	1940 / 480	$< 10^{-200}$
SmolLM2-135M − ours	+4.16 [+3.6, +4.8]	1532 / 871	8×10^{-42}
GPT-2 − ours	+0.91 [+0.3, +1.5]	1287 / 1142	0.0035
Pythia-160M − ours	−17.23 [−18.0, −16.5]	647 / 3383	$< 10^{-300}$
Ours − no pretraining	+40.62 [+39.8, +41.4]	6704 / 254	$< 10^{-300}$

Table 4: **Paired significance tests** on test (seed 0). "Discordant" counts questions only the first system answers correctly, then questions only the second does.

2. **About one point behind GPT-2.** GPT-2 is ahead by 0.9 points at greedy ($p = 0.004$) and 2.0 points with both constraints. On dev the greedy gap was 0.5 points and not significant ($p = 0.26$), so we report the test result: GPT-2 is slightly better.

3. **Far ahead of Pythia-160M,** by 17 points at greedy, despite Pythia's 67× larger pretraining budget. Pythia invents column names far more often (8.5% of test questions produce invalid SQL, vs 2.0% for ours). Our ablation (Section 5.3) suggests that the SQL-heavy mix explains only a small part of this gap, so we do not claim a cause.

4. **Scale still wins at equal size:** SmolLM2-135M, with about 440× our pretraining tokens, is 4.6 points ahead (mean over seeds), and SmolLM2-360M is 9.1 points ahead.

5. **Results are stable.** Seed standard deviations are 0.1–0.4 points, far smaller than every gap between models. Our dev-to-test drop is 0.7 points, similar to the baselines' 0.4–0.5 (Pythia, whose fine-tuning is unstable, drops 5 points).

5.2 Constrained decoding

Constraints raise validity to 100% for every model and help weaker models more (Table 5, Figure 8).

The value constraint is worth more than the schema constraint for every model: copying filter values exactly is where small models slip most. For our model, constrained decoding *never* turned a correct answer into a wrong one in any paired comparison we ran (0 broken on test at both levels; 0 on dev). For every baseline it sometimes does, plausibly because their tokenizers can be forced into token splits they rarely produce; we did not verify this.

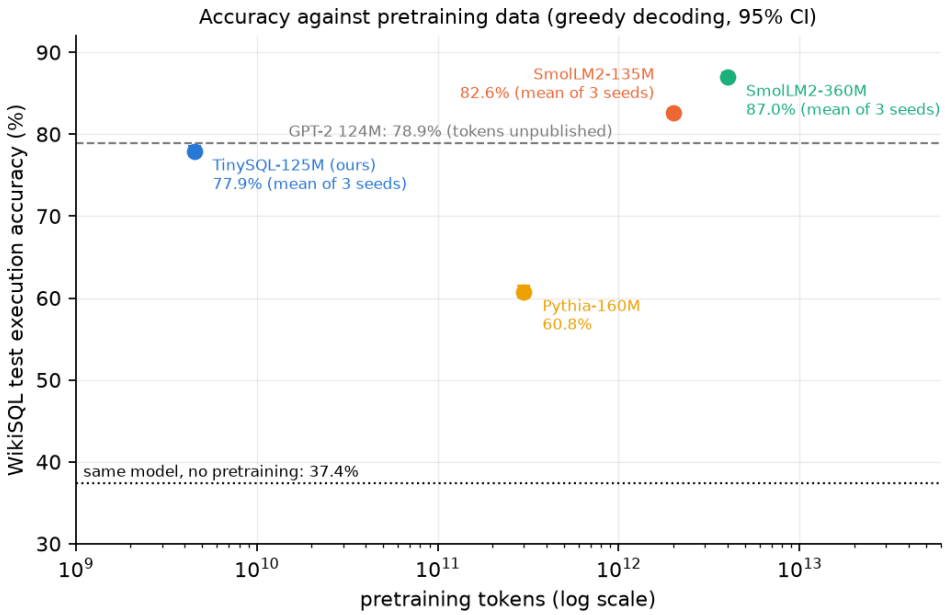


Figure 7: **Accuracy vs. pretraining data:** test execution accuracy (greedy, 95% CI) against pretraining tokens. GPT-2's token count is not published, so it is drawn as a reference line, as is our no-pretraining control.

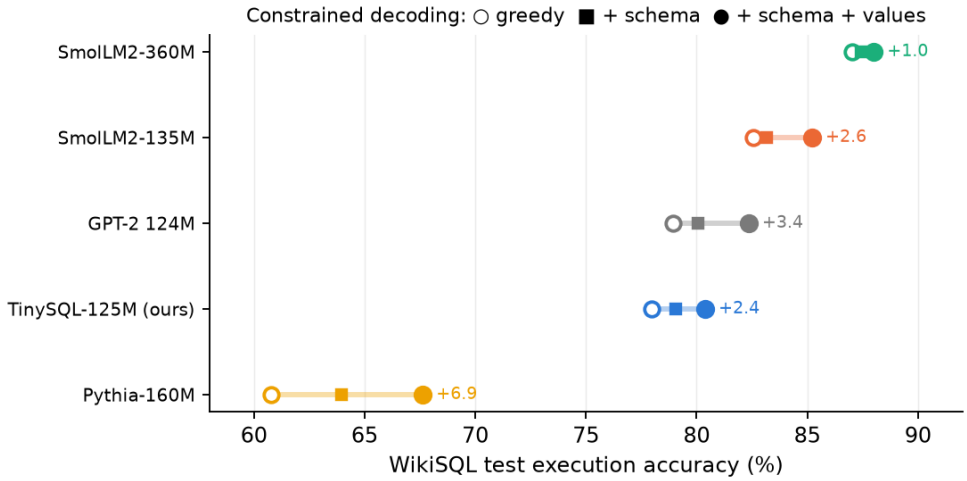


Figure 8: **Gains from constrained decoding:** test accuracy with greedy decoding (open circle), the schema constraint (square) and schema + values (filled circle); mean over seeds where available. The gain shrinks as the model gets stronger.

Model	+ Schema	+ Schema + values	Fixed / broken (schema + values)
TinySQL-125M (ours)	+1.05	+2.42	385 / **0**
GPT-2 124M	+1.14	+3.43	659 / 114
Pythia-160M	+3.19	+6.85	1495 / 407
SmolLM2-135M	+0.98	+2.81	483 / 37
SmolLM2-360M	+0.21	+0.94	189 / 40
No pretraining	–	+10.94	1755 / 18

Table 5: **Gains from constrained decoding** over greedy decoding with the same weights (test, seed 0, points; all $p < 0.005$). "Fixed" and "broken" count questions that change from wrong to right and from right to wrong.

Variant (one change from base)	Params	Dev accuracy	Δ	p
Base (RoPE, GQA 6/2, tied, mixed data)	25.2M	**55.82** [54.8, 56.9]		
Learned absolute positions instead of RoPE	25.6M	47.35	−8.48	7×10^{-89}
Untied input/output embeddings	37.8M	50.83	−5.00	6×10^{-32}
Full multi-head attention	26.7M	55.08	−0.75	0.055
Web text only (no SQL, code or Q&A)	25.2M	54.89	−0.94	0.021

Table 6: **Ablations** on a 25M-parameter proxy (8 layers, $d = 384$, context 1024), each pretrained on 150M tokens and fine-tuned with the same recipe. Greedy dev execution accuracy, one seed each; p from McNemar against the base.

5.3 Ablations

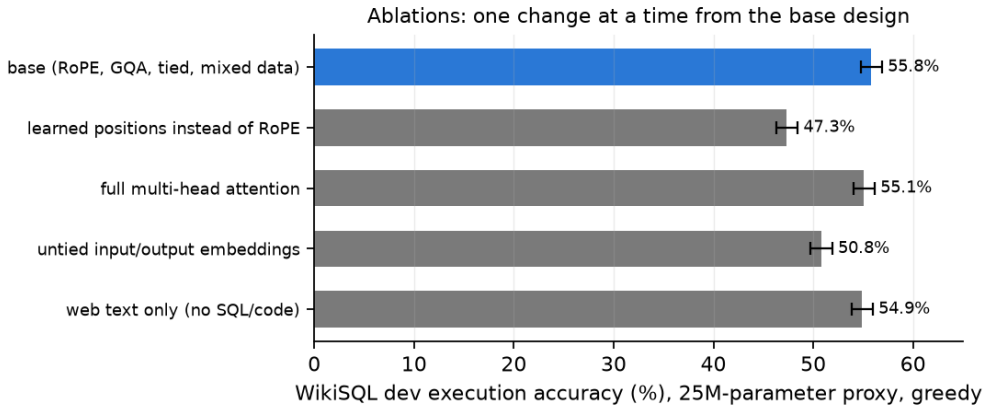


Figure 9: **Ablations** on the 25M proxy: one design change at a time from the base model (dev, greedy, 95% CI).

Table 6 and Figure 9 vary one design choice at a time on a small proxy model. Rotary embeddings matter most: without them, accuracy drops 8.5 points and pretraining loss is clearly worse (4.51 vs 4.09 nats on web text). Tying embeddings helps by 5 points even though the untied model has 50% more parameters. Grouped-query attention costs nothing measurable while shrinking the KV cache 3×. Most surprisingly, removing all SQL, code and Q&A data from pretraining costs only 0.9 points after fine-tuning, even though the web-only model's loss on SQL code is far higher (4.40 vs 1.81 nats). Fine-tuning on 56k WikiSQL examples appears to recover most of what the

SQL data teaches. These are single-seed results at 150M tokens, so we treat them as indicative.

5.4 Failure analysis

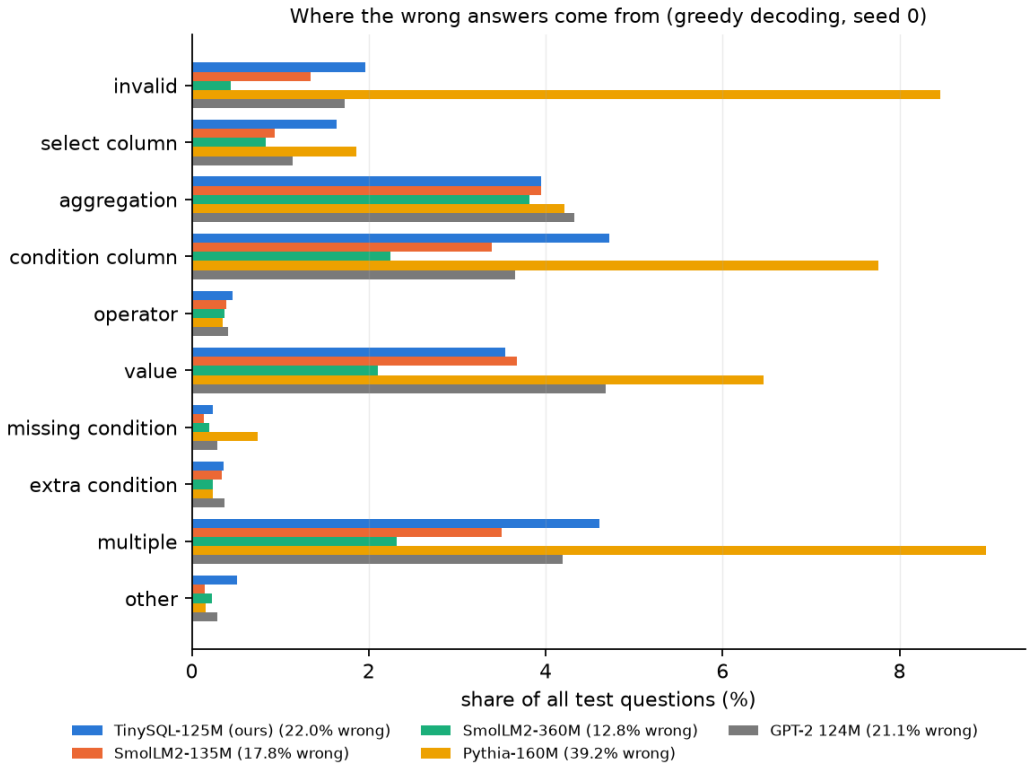


Figure 10: **Failure categories:** where wrong answers come from (test, greedy, seed 0). Each wrong prediction is parsed into WikiSQL's structured form and assigned exactly one category.

Each wrong prediction is parsed back into WikiSQL's structure (selected column, aggregation, conditions) and compared with gold, giving exactly one category (Figure 10). Compared with stronger models, our model loses most on condition columns, values and multi-part errors, the skills that depend on reading the table and the question. Aggregation errors are different: they stay at 3.8–4.3% of test questions for every pretrained model, although overall accuracy spans 26 points.

Query complexity. Accuracy falls as the gold query needs more conditions (Figure 11). On the 10,832 test questions with a single condition, our model and GPT-2 are level (80.3% and 80.2%, seed 0). The gap between them opens with two conditions (73.5% vs 76.2% on 3,879 questions), which matches the multi-part errors in Figure 10. Pythia falls fastest, from 64.5% to 35.8% at four conditions.

Aggregation label audit. To find out why, we hand-checked 50 randomly sampled dev questions where our model was wrong *only* in the aggregation. In 31 (62%; 95% Wilson CI [24] 48–74%) the gold aggregation is not supported by the question. For example, "How many people were in the crowd when Essendon was the home team?" is labelled `COUNT(Crowd)` rather than the crowd figure,

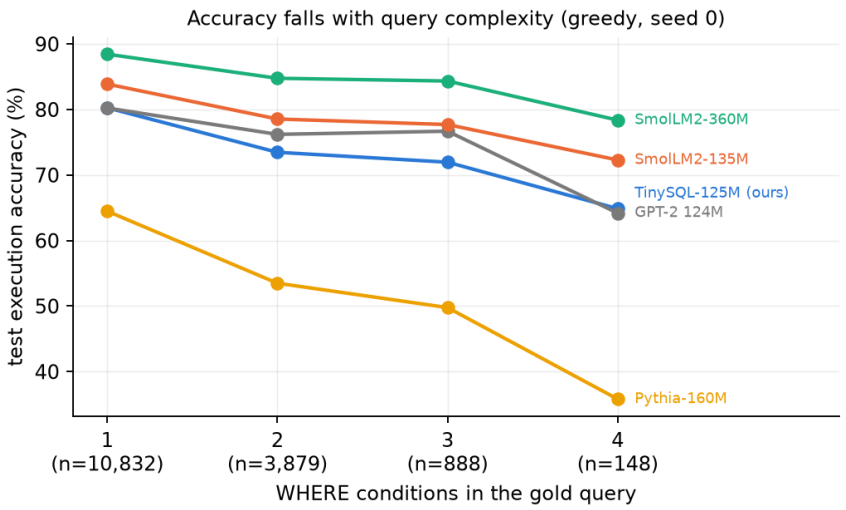


Figure 11: **Accuracy by query complexity:** test accuracy by the number of WHERE conditions in the gold query (greedy, seed 0). The 131 questions without conditions are omitted.

and "For record 25-24, what is the sum of attendance?" is labelled `COUNT`. A further 12 (24%) are ambiguous, and only 7 (14%) are clear model errors. Extrapolated, roughly 2 points of dev accuracy are lost to aggregation labels for every model, which is consistent with earlier reports of annotation errors in WikiSQL [9, 15]. The audit had a single annotator (the author), not blinded; all 50 judgements are released for re-checking.

6 Limitations

- **One benchmark, one query shape.** WikiSQL queries are single-table with no joins, grouping or nesting. We do not evaluate on harder benchmarks such as Spider [25].
- **Uncontrolled baseline pretraining.** The baselines differ in pretraining data, tokenizers and architecture, not just in token count, so Figure 7 shows association, not a scaling law.
- **One pretraining run.** Seeds vary fine-tuning only. GPT-2, Pythia, the control and all ablations have a single fine-tuning seed.
- **Noisy selection.** Learning rates and epochs are selected on 1,000 dev questions, whose 95% CI is about ±2.8 points, wider than the differences between adjacent learning rates.
- **Reproducibility on GPU.** Fine-tuning is not bit-reproducible on GPU even with fixed seeds. Pythia's re-fine-tune for test, for example, differs from its dev run.
- **Decontamination** covers dev/test questions of 8 or more words, not short questions or SQL strings. The audit is single-annotator.

7 Conclusion

A single developer can build every component of a text-to-SQL language model, pretrain it on 4.5B tokens in two days on one GPU, and land within about one point of GPT-2 on WikiSQL while

outperforming a model pretrained on 67× more data. Constrained decoding is a cheap and safe addition at this scale. Getting the last points still requires much more pretraining, and on WikiSQL a measurable share of the remaining errors are in the labels rather than in the models.

Reproducibility

Code, configurations, all prediction-level results, statistics and figures are at https://github.com/aditya0929/tinysql. `PROJECT_LOG.md` records every run, every number and every bug found along the way. Total compute was about 46 L4 GPU-hours for pretraining and about 45 for the benchmark (fine-tuning, evaluation and ablations), all on NVIDIA L4 GPUs.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, 2023.

[2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, et al. SmolLM2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

[3] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, 2023.

[4] Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences*, 1997.

[5] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of ACL*, 2018.

[6] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[7] Gretel AI. synthetic_text_to_sql. https://huggingface.co/datasets/gretelai/synthetic_text_to_sql, 2024.

[8] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[9] Wonseok Hwang, Jinyeong Yim, Seunghyun Park, and Minjoon Seo. A comprehensive exploration on WikiSQL with table-aware word contextualization. *arXiv preprint arXiv:1902.01069*, 2019.

[10] Denis Kocetkov, Raymond Li, Loubna Ben Allal, et al. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[11] Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of EMNLP*, 2004.

[12] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[13] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[14] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[15] Dzmitry Pihulski, Karol Charchut, Viktoria Novogrodskaia, and Jan Kocoń. LLMSQL: Upgrading WikiSQL for the LLM era of text-to-SQL. *arXiv preprint arXiv:2510.02350*, 2025.

[16] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of EACL*, 2017.

[17] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[18] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of EMNLP*, 2021.

[19] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of ACL*, 2016.

[20] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[21] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[22] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[23] Chenglong Wang, Kedar Tatwawadi, Marc Brockschmidt, Po-Sen Huang, Yi Mao, Oleksandr Polozov, and Rishabh Singh. Robust text-to-SQL generation with execution-guided decoding. *arXiv preprint arXiv:1807.03100*, 2018.

[24] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[25] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of EMNLP*, 2018.

[26] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, 2019.

[27] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2SQL: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.

A Additional results

Model	5×10^{-5}	10^{-4}	2×10^{-4}	Chosen
TinySQL-125M	74.3	73.4	68.8	5×10^{-5}
SmolLM2-135M	78.0	78.3	75.1	10^{-4}
SmolLM2-360M	84.6	84.5	80.5	5×10^{-5}
GPT-2 124M	72.4	75.2	74.5	10^{-4}
Pythia-160M	56.9	58.5	45.7	10^{-4}

Table 7: **Learning-rate sweep (numbers):** execution accuracy on the first 1,000 dev questions, best epoch, seed 0. Control: 28.6 (10^{-4}) vs 11.5 (10^{-3}).

Model	Greedy	+ Schema	+ Schema + values
TinySQL-125M (ours)	78.63 ± 0.26	79.68*	80.96 ± 0.21
GPT-2 124M	79.41	80.60	82.79
Pythia-160M	65.94	68.38	72.15
SmolLM2-135M	82.97 ± 0.22	83.46*	85.45 ± 0.22
SmolLM2-360M	87.40 ± 0.20	87.55*	88.41 ± 0.10
No pretraining	37.70	–	48.22

Table 8: **Dev results:** WikiSQL **dev** execution accuracy (%, 8,421 questions). *Single run at learning rate 10^{-4}.

	Execution		Logical form	
Model	Greedy	+ Schema + values	Greedy	+ Schema + values
TinySQL-125M (ours)	78.03 ± 0.19	80.50 ± 0.20	72.28 ± 0.09	74.48 ± 0.07
GPT-2 124M	78.94	82.35	73.25	76.41
Pythia-160M	61.02	68.04	55.10	61.17
SmolLM2-135M	82.61 ± 0.35	85.27 ± 0.19	77.42 ± 0.39	79.91 ± 0.25
SmolLM2-360M	87.00 ± 0.17	87.99 ± 0.10	81.71 ± 0.17	82.66 ± 0.10
No pretraining	37.64	48.46	31.60	40.76

Table 9: **Official evaluator:** WikiSQL **test** scores from Salesforce's evaluate.py (%). 95% bootstrap CIs (our executor) for the greedy rows: ours [77.4, 78.5], GPT-2 [78.3, 79.6], Pythia [60.0, 61.5], SmolLM2-135M [82.0, 83.1], SmolLM2-360M [86.6, 87.5], no pretraining [36.7, 38.1].